Preconditioning Hints for Time Series Forecasting

Benjamin Cole
Advisor: Elad Hazan
Second Reader: John Sheridan

April 28, 2026

1 Introduction

At a high level, the objective of time series analysis is to predict the next element in a sequence as accurately as possible. The ability to do so has far-reaching ramifications, from medicine to finance to meteorology to, more recently, language modeling. One classical approach, first analyzed by Box and Jenkins in the 1970's [2], is differencing; that is, given an online sequence $y_0, y_1, ..., y_t$, instead of predicting y_{t+1}, we predict $y_{t+1} - y_t$ using $y_1 - y_0, y_2 - y_1, ...y_t - y_{t-1}$, which is an easier task under certain conditions.

In 2025, based on the theory of linear dynamical systems, Marsden and Hazan [7] proposed universal sequence preconditioning (USP) for time series prediction. The technique generalizes the notion of differencing, again tackling the problem at hand not by improving the machine that creates the outputs, but by judiciously combining previously observed outputs of the system. In particular, they suggest preprocessing the inputs via a fixed convolution of polynomial coefficients, and prove that this can result in hidden-dimension-free (up to logarithmic factors) sublinear regret with respect to the best LDS predictor. The basic approach from the paper is below, in Algorithm 1.

Note that the procedure is quite simple; it involves nothing but convolving the input with coefficients of a fixed polynomial, running a preexisting algorithm on the preconditioned sequence, and then unpreconditioning by reversing the convolution in order to transform the outputs of the model back into the original space. USP is universal for two reasons: 1) The polynomial, if chosen correctly, can work on a wide range of inputs. 2) It is intended to be algorithm-agnostic. In the paper, Marsden and Hazan mostly use the nth degree monic Chebyshev polynomial, and specialize to a few particular algorithms to prove regret guarantees.

Presented in this manner, USP improves prediction accuracy under the assumption that when we unprecondition, ground truth $y_{t-1}, y_{t-2}, ..., y_{t-n}$ has already been revealed to us (see line 11 of Algorithm 1). While Marsden and Hazan brush this under the rug, it is no small assumption. Modern time series analysis quickly loses its appeal if we are limited to

Algorithm 1 Universal Sequence Preconditioning (Offline Version) [7]

Training
Input: training data $(u_{1:T}^{1:N}, y_{1:T}^{1:N})$ where (u_t^i, y_t^i) is the t-th input/output pair in the i-th example sequence; preconditioning coefficients $c_{0:n}$; algorithm $\mathcal{A}$.
Assert that $c_0 = 1$.
for $i = 1$ to N **do**
    $y_{1:T}^{\text{preconditioned},i} \leftarrow \texttt{convolution}(y_{1:T}^i, c_{0:n})$ $\quad \triangleleft\ y_t^{\text{preconditioned},i} = y_t^i + \sum_{j=1}^n c_j y_{t-j}^i$
end for
Train algorithm $\mathcal{A}$ on preconditioned training data $\left(u_{1:T}^{1:N}, y_{1:T}^{\text{preconditioned},1:N}\right)$.
Test Time
for $t = 1$ to T **do**
    Receive u_t.
    Output prediction of y_t as $\hat{y}_t \leftarrow \mathcal{A}\left(u_{1:t}, y_{1:(t-1)}\right) - \sum_{i=1}^n c_i y_{t-i}$.
    Receive y_t.
end for

predicting one step ahead. Suppose, for instance, that we were meteorologists who predicted weather based on a historical dataset with time increments of 30 minutes. We would be entirely useless if we could only forecast the weather half an hour in advance!

With this in mind, in the rest of the paper, we will establish the necessary mathematical background; prove a regret guarantee for preconditioning; explain why other methods fail; understand when preconditioning also fails in the next-step-ahead case; explore the multi-step forecasting problem described above; present a solution to the problem; and provide empirical results.

2 Background

A noiseless linear dynamical system follows

$$\begin{aligned} x_{t+1} &= Ax_t + Bu_t \\ y_t &= Cx_t \end{aligned} \tag{1}$$

for inputs $u_0, ..., u_T \in \mathbb{R}^{d_{in}}$, hidden states $x_0, ..., x_T \in \mathbb{R}^{d_{hidden}}$, and observations $y_0, ..., y_T \in \mathbb{R}^{d_{out}}$. We will assume throughout that the spectral radius, i.e., the magnitude of the largest eigenvalue, of A satisfies $\rho(A) \leq 1$ and that $||u_t||_2 \leq 1\ \forall t$, so as to ensure that the system is stable.

Over the past few decades, various methods have been proposed to predict the next observation of an LDS given that the underlying dynamics are unknown, all subject to their own difficulties.

Attempting to directly learn A, B, and C, called system identification, is a nonconvex problem and fails if the system is marginally stable, which for our purposes we will take to mean if $\rho(A) = 1$. (Having $\rho(A) = 1$ does not always imply marginal stability, but we will assume diagonalizability later on, which is a sufficient condition for this to hold.)

Open-loop predictors, of the form $\hat{y}_t = \sum_{j=0}^{n} Q_j u_{t-j}$, utilize a truncated sequence of past inputs in order to predict; however, this again becomes impractical in the case of a marginally stable system.

Closed-loop predictors are similar to their open-loop counterparts, except that they incorporate past observations as well. In particular, they satisfy $\hat{y}_t = \sum_{j=0}^{n} Q_j u_{t-j} + \sum_{i=1}^{m} R_i y_{t-i}$, and can successfully learn the dynamics of the system with zero error when $n = m = d_{hidden}$ due to the Cayley-Hamilton theorem. The issue in this case is thus not the need for extra restrictions on A, but rather that the hidden dimension can be arbitrarily large.

Spectral filtering, proposed by Hazan, Singh, and Zhang in 2017 [5], involves convolving the past inputs with the top eigenvectors of a fixed Hankel matrix. It circumvents the marginal stability and hidden dimension dependence problems, but needs A to be symmetric or diagonalizable over $\mathbb{R}$.

Finally, USP [7] allows for learning the LDS independently of the hidden dimension, even if the system is marginally stable, as long as the imaginary components of the eigenvalues of A are bounded from above. In fact, the method is nearly tight, in the sense that it is provably true that for large enough imaginary components of the eigenvalues, any learning method must depend on d_{hidden}.

In this paper, we will work in the online setting, meaning that at each time step, we predict $\hat{y}_t$, observe the true y_t, suffer a loss $\ell(\hat{y}_t, y_t)$, and then update our parameters based on the loss. This is to be contrasted with offline learning, in which we already start with a dataset before we make any predictions. In Online Convex Optimization (OCO) [3], the primary tool we use to measure the strength of an algorithm is regret. For a convex set $\mathcal{K} \subseteq \mathbb{R}^n$ and adversarially chosen convex loss functions $f_t : \mathcal{K} \to \mathbb{R}$, suppose that an algorithm $\mathcal{A}$ gives us our current moves based on the game history:

$$x_t^{\mathcal{A}} = \mathcal{A}(f_1, ..., f_{t-1}) \in \mathcal{K}$$

From this, we define the regret of $\mathcal{A}$ after T iterations to be

$$\text{Regret}_T = \sup_{f_1,...,f_T} \left\{ \sum_{t=1}^{T} f_t(x_t) - \min_{x \in \mathcal{K}} \sum_{t=1}^{T} f_t(x) \right\}$$

In other words, because the loss functions can be chosen adversarially, simply trying to minimize the sum of losses is a futile exercise; rather, we attempt to minimize the sum of losses relative to the best fixed move in hindsight. Generally, the goal is to achieve sublinear regret in T, because this implies that we perform as well as the best fixed move in hindsight as $T \to \infty$. The primary sublinear regret algorithm we will leverage later on is online gradient descent, stated in Algorithm 2.

Theorem 2.1. *Let D be the diameter of the decision set $\mathcal{K}$ and let G be an upper bound on the Lipschitz constant of the cost functions f_t. Then $\forall T \geq 1$, online gradient descent with step sizes $\eta_t = \frac{D}{G\sqrt{t}}$ guarantees that*

$$\textit{Regret}_T \leq \frac{3}{2} DG\sqrt{T}$$

Algorithm 2 Online Gradient Descent [3]

Input: convex set $\mathcal{K}$, T, $x_1 \in \mathcal{K}$, step sizes $\{\eta_t\}$
for $t = 1$ to T **do**
    Play x_t and observe cost $f_t(x_t)$.
    Update and project:

$$y_{t+1} = x_t - \eta_t \nabla f_t(x_t)$$

$$x_{t+1} = \arg\min_{x \in \mathcal{K}} \|x - y_{t+1}\|^2 = \prod_{\mathcal{K}} [y_{t+1}]$$

end for

Proving this statement is a matter of manipulating the definitions of convexity and projection, and is beyond the scope of this paper. As such, we will take it as a given.

Note that we cannot apply OGD directly to achieve sublinear regret with respect to the predictor class of all linear dynamical systems, as this is a nonconvex set. However, what we can and will do is choose a convex predictor class, and then prove that the approximation error is small enough such that the total bound is still sublinear.

3 Why USP Works For Next Step Prediction

We now present a simplified exposition of the regret bound given in Marsden and Hazan [7], which helps in understanding the basics of USP.

The importance of the Chebyshev polynomial lies in the following two lemmas, which we will then use to demonstrate how USP achieves sublinear regret.

Lemma 3.1. *The nth degree monic Chebyshev polynomial $M_n(x)$ satisfies*

$$\max_{x \in [-1,1]} |M_n(x)| = 2^{-(n-1)}$$

Proof. Recall that the nth degree (nonmonic) Chebyshev polynomial $T_n(x)$ can be formulated in two distinct, yet equivalent, ways. The first is

$$T_n(x) = \cos(n \arccos(x))$$

for $x \in [-1, 1]$. The second is by the recurrence

$$T_0(x) = 1,\ T_1(x) = x,\ T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$$

It is clear from the latter by an inductive argument that the leading coefficient of $T_n(x)$ increases exponentially as $2^{(n-1)}$ for $n \geq 1$. Thus, we have that $M_n(x) = \frac{T_n(x)}{2^{(n-1)}}$. Substituting in the former formulation, this implies that

$$|M_n(x)| = \left|\frac{T_n(x)}{2^{(n-1)}}\right| = \left|\frac{\cos(n \arccos(x))}{2^{(n-1)}}\right| \leq \frac{1}{2^{(n-1)}}$$

with the inequality becoming an equality when $x = \cos\left(\frac{k\pi}{n}\right)$ for $k = 0, 1, ..., n$. □

In fact, $M_n(x)$ is the monic polynomial of degree n that minimizes its absolute maximum on the interval $[-1, 1]$. This is the fundamental minimax property of the Chebyshev polynomial.

Lemma 3.2. *If $c_0, c_1, ..., c_n$ are the coefficients of $M_n(x)$, then*

$$\max_k |c_k| \leq 2^{0.3n+1}$$

Proof. Yet another way of writing $M_n(x)$ is

$$M_n(x) = n \sum_{k=0}^{\lfloor \frac{n}{2} \rfloor} (-1)^k \frac{(n-k-1)!}{k!(n-2k)!} 2^{-2k} x^{n-2k}$$

Since $k \leq \frac{n}{2}$, we know that $n \leq 2(n-k)$, and so

$$|c_k| = n \frac{(n-k-1)!}{k!(n-2k)!} 4^{-k} \leq 2 \frac{(n-k)!}{k!(n-2k)!} 4^{-k} \leq 2 \frac{(n-k)^k}{k!} 4^{-k}$$

$$\leq 2 \frac{(n-k)^k}{(k/e)^k} 4^{-k} = 2 \left(\frac{(n-k)e}{4k} \right)^k$$

Letting $k = cn$, we can convert the problem to a continuous domain:

$$\max_{k=0,1,\ldots,n} |c_k| \leq \max_{k=0,1,\ldots,n} 2 \left(\frac{(n-k)e}{4k} \right)^k \leq \max_{c \in [0,1]} 2 \left(\frac{(1-c)e}{4c} \right)^{cn}$$

$$\leq 2 \cdot 2^{0.3n} = 2^{0.3n+1}$$

where the last inequality follows numerically. □

Algorithm 3 Universal Sequence Preconditioning for Regression [7]

Input: initial parameter Q^0; preconditioning coefficients $c_{0:n}$ from the n-th degree (monic) Chebyshev polynomial; convex constraints $\mathcal{K} = \{(Q_0, \ldots, Q_{n-1}) \text{ s.t. } \|Q_j\| \leq C_{\text{domain}} \|c\|_1\}$
Assert that $c_0 = 1$.
for $t = 1$ **to** T **do**
    Receive u_t.
    Predict $\hat{y}_t(Q^t) = -\sum_{i=1}^n c_i y_{t-i} + \sum_{j=0}^n Q_j^t u_{t-j}$.
    Observe true output y_t and suffer loss $\ell_t(Q^t) = \|\hat{y}_t(Q^t) - y_t\|_1$.
    Update and project:

$$Q^{t+1} \leftarrow \text{proj}_{\mathcal{K}} \left(Q^t - \eta_t \nabla_Q \ell_t(Q^t) \right).$$

end for

Now, we have the tools to analyze convex regression with preconditioning by the coefficients of the Chebyshev polynomial, as described in Algorithm 3. Keep in mind that USP is

intended to be universal (see Algorithm 1), but we specialize the algorithm here so that we can prove a regret bound. In any case, we will prove a somewhat weaker version of the original theorem in the USP paper for brevity; we will assume that A is diagonalizable over $\mathbb{R}$, instead of diagonalizable over $\mathbb{C}$ with its eigenvalues having a particular bound on their imaginary components.

Theorem 3.3. *Consider the linear dynamical system 1 defined above. Assume that $\rho(A) \leq 1$ and that $||u_t||_2 \leq 1$ $\forall t$ for system stability. Additionally, assume that A is diagonalizable over $\mathbb{R}$, i.e., $A = PDP^{-1}$ with P and D being real-valued. Defining the condition number of A to be $\kappa = ||P||_2||P^{-1}||_2$, assume that $||C||_F||B||_2\kappa \leq C_{domain}$. Then Algorithm 3 satisfies*

$$\sum_{t=1}^{T} \ell_t(Q^t) = \sum_{t=1}^{T} ||\hat{y}_t - y_t||_1 \leq C_{domain}\sqrt{d_{out}}\left(3n\,||c||_1\sqrt{T} + \max_{\lambda\in\lambda(A)} |p_n(\lambda)|(T-n)^2\right)$$

Proof. Unrolling the LDS gives us the form

$$y_t = \sum_{s=1}^{t} CA^{t-s}Bu_s$$

If we define $p_n(x) = \sum_{i=0}^{n} c_i x^{n-i}$, rearranging gives us

$$y_t = -\sum_{i=1}^{n} c_i y_{t-i} + \sum_{s=0}^{n-1}\sum_{i=0}^{s} c_i CA^{s-i}Bu_{t-s} + \sum_{s=0}^{t-n-1} Cp_n(A)A^sBu_{t-n-s}$$

where the first term is a byproduct of the preconditioning, the second term is the effect of the recent past on the current observation, and the third term is the effect of the more distant past on the observation. Now, consider the predictor $\hat{Q} = (\hat{Q}_0, ..., \hat{Q}_{n-1})$, where $\hat{Q}_i = \sum_{j=0}^{i} c_j CA^{i-j}B$. The idea is that we plucked this $\hat{Q}$ from the recent past term in the above equation, and we will argue that we can effectively learn this term via OCO. On the other hand, we can make the distant past term small enough via a judicious choice of coefficients (which will be those of the monic Chebyshev polynomial). Since

$$||\hat{Q}_i||_F = \left|\left|\sum_{j=0}^{i} c_j CA^{i-j}B\right|\right|_F \leq \sum_{j=0}^{i} |c_j|||C||_F||A^{i-j}||_2||B||_2$$
$$\leq \kappa||C||_F||B||_2\,||c||_1 \leq C_{domain}\,||c||_1$$

we have that $\hat{Q} \in \mathcal{K}$. It follows that

$$\min_{Q^*\in\mathcal{K}} \sum_{t=1}^{T} \ell_t(Q^*) \leq \sum_{t=1}^{T} \ell_t(\hat{Q}) = \sum_{t=1}^{T} ||\hat{y}_t(\hat{Q}) - y_t||_1$$

where

$$||\hat{y}_t(\hat{Q}) - y_t||_1$$
$$= \left|\left|\left(-\sum_{i=1}^{n} c_i y_{t-i} + \sum_{j=0}^{n-1} \hat{Q}_j u_{t-j}\right) - \left(-\sum_{i=1}^{n} c_i y_{t-i} + \sum_{s=0}^{n-1}\sum_{i=0}^{s} c_i CA^{s-i}Bu_{t-s} + \sum_{s=0}^{t-n-1} Cp_n(A)A^sBu_{t-n-s}\right)\right|\right|_1$$

$$= \left|\left| - \sum_{s=0}^{t-n-1} Cp_n(A)A^s Bu_{t-n-s} \right|\right|_1 \leq \sqrt{d_{out}} \sum_{s=0}^{t-n-1} ||Cp_n(A)A^s Bu_{t-n-s}||_2$$

$$\leq \sqrt{d_{out}} \cdot (t-n) \cdot \max_s ||Cp_n(A)A^s B||_2 = \sqrt{d_{out}} \cdot (t-n) \cdot \max_s ||CPp_n(D)D^s P^{-1} B||_2$$

$$\leq \sqrt{d_{out}} \max_{\lambda \in \lambda(A)} |p_n(\lambda)| ||C||_2 ||B||_2 \kappa(t-n) \leq \sqrt{d_{out}} \max_{\lambda \in \lambda(A)} |p_n(\lambda)| C_{domain}(t-n)$$

Continuing the thread of algebra from above and summing over t, this implies that

$$\min_{Q^* \in \mathcal{K}} \sum_{t=1}^{T} \ell_t(Q^*) \leq \sum_{t=1}^{T} ||\hat{y}_t(\hat{Q}) - y_t||_1 \leq \sqrt{d_{out}} \max_{\lambda \in \lambda(A)} |p_n(\lambda)| C_{domain} \sum_{t=n+1}^{T} (t-n)$$

$$= \sqrt{d_{out}} \max_{\lambda \in \lambda(A)} |p_n(\lambda)| C_{domain} \cdot \frac{(T-n)(T-n+1)}{2} \leq \sqrt{d_{out}} \max_{\lambda \in \lambda(A)} |p_n(\lambda)| C_{domain}(T-n)^2$$

Next, we will bound the regret of Algorithm 3 with respect to the best fixed predictor in our convex set. By Theorem 2.1, OGD gives us the bound

$$\text{Regret}_T = \sum_{t=1}^{T} \ell_t(Q^t) - \min_{Q^* \in \mathcal{K}} \sum_{t=1}^{T} \ell_t(Q^*) \leq \frac{3}{2} DG\sqrt{T}$$

for diameter D of $\mathcal{K}$ and gradient norm bound G of ℓ_t. To bound D, observe that

$$D = \max_{Q^1, Q^2 \in \mathcal{K}} ||Q^1 - Q^2||_F \leq \sqrt{\sum_{j=0}^{n-1} (2C_{domain} ||c||_1)^2} = 2\sqrt{n} C_{domain} ||c||_1$$

For G, recall that

$$\ell_t(Q^t) = \left|\left| - \sum_{i=1}^{n} c_i y_{t-i} + \sum_{j=0}^{n} Q_j^t u_{t-j} - y_t \right|\right|_1$$

and so

$$\left|\left|\nabla_{Q_i^t} \ell_t(Q_i^t)\right|\right|_F = \left|\left|\text{sign}(\hat{y}_t - y_t) u_{t-i}^T\right|\right|_F = ||\text{sign}(\hat{y}_t - y_t)||_2 \, ||u_{t-i}||_2 \leq \sqrt{d_{out}} \cdot 1 = \sqrt{d_{out}}$$

It follows that the total gradient norm is bounded by

$$G = \max_{t \in [T]} \left|\left|\nabla_Q \ell_t(Q^t)\right|\right|_F \leq \sqrt{\sum_{j=0}^{n-1} (\sqrt{d_{out}})^2} = \sqrt{n d_{out}}$$

Plugging these figures into our OGD bound yields

$$\sum_{t=1}^{T} \ell_t(Q^t) - \min_{Q^* \in \mathcal{K}} \sum_{t=1}^{T} \ell_t(Q^*) \leq 3C_{domain} n \sqrt{d_{out}} \, ||c||_1 \sqrt{T}$$

At this point, we can combine our regret and fixed Q^* bounds to achieve a bound of

$$\sum_{t=1}^{T} \ell_t(Q^t) = \sum_{t=1}^{T} \ell_t(Q^t) - \min_{Q^* \in \mathcal{K}} \sum_{t=1}^{T} \ell_t(Q^*) + \min_{Q^* \in \mathcal{K}} \sum_{t=1}^{T} \ell_t(Q^*)$$

$$\leq 3C_{domain} n \sqrt{d_{out}} \, ||c||_1 \sqrt{T} + \sqrt{d_{out}} \max_{\lambda \in \lambda(A)} |p_n(\lambda)| C_{domain} (T-n)^2$$

$$= C_{domain} \sqrt{d_{out}} \left(3n \, ||c||_1 \sqrt{T} + \max_{\lambda \in \lambda(A)} |p_n(\lambda)| (T-n)^2 \right)$$

as desired. □

Theorem 3.4. *For the nth degree monic Chebyshev polynomial, Algorithm 3 satisfies*

$$\frac{1}{T} \sum_{t=1}^{T} \ell_t(Q^t) = \tilde{O}\left(T^{-\frac{2}{13}}\right)$$

where $\tilde{O}(\cdot)$ hides polylogarithmic factors in T.

Proof. By Theorem 3.3, we have that

$$\sum_{t=1}^{T} \ell_t(Q^t) \leq C_{domain} \sqrt{d_{out}} \left(3n \, ||c||_1 \sqrt{T} + \max_{\lambda \in \lambda(A)} |p_n(\lambda)| (T-n)^2 \right)$$

$$\leq C_{domain} \sqrt{d_{out}} \left(3n(n+1) \cdot 2^{0.3n+1} \cdot \sqrt{T} + 2^{-(n-1)} \cdot (T-n)^2 \right)$$

where in the second inequality we applied Lemmas 3.1 and 3.2 to specialize this bound to Chebyshev. Choosing $n = \left\lceil \frac{15}{13} \log_2 T \right\rceil$, we get for the first term

$$3n(n+1) \cdot 2^{0.3n+1} \cdot \sqrt{T} \leq 3 \left(\frac{15}{13} \log_2 T + 2 \right)^2 \cdot 2^{0.3\left(\frac{15}{13} \log_2 T + 1\right) + 1} \cdot \sqrt{T}$$

$$= O\left(\log_2^2 T \cdot T^{\frac{4.5}{13}} \cdot T^{\frac{1}{2}} \right) = O\left(\log_2^2 T \cdot T^{\frac{11}{13}} \right) = \tilde{O}\left(T^{\frac{11}{13}} \right)$$

Similarly, for the second term, since $(T-n)^2 \leq T^2$,

$$2^{-(n-1)} \cdot (T-n)^2 \leq 2^{-(n-1)} \cdot T^2 \leq 2^{-\left(\frac{15}{13} \log_2 T - 1\right)} \cdot T^2$$

$$= O\left(T^{-\frac{15}{13}} \cdot T^2 \right) = O\left(T^{\frac{11}{13}} \right)$$

Substituting these results into our bound for the cumulative loss yields

$$\sum_{t=1}^{T} \ell_t(Q^t) \leq C_{domain} \sqrt{d_{out}} \left(\tilde{O}\left(T^{\frac{11}{13}} \right) + O\left(T^{\frac{11}{13}} \right) \right) = \tilde{O}\left(T^{\frac{11}{13}} \right)$$

and thus

$$\frac{1}{T} \sum_{t=1}^{T} \ell_t(Q^t) = \tilde{O}\left(T^{-\frac{2}{13}} \right)$$

Hence, applying USP to convex regression indeed gives us the sublinear regret bound that we were searching for. □

4 Why Other Approaches Fail

In this section, we will build some intuition as to why USP is useful by investigating the failure modes of open- and closed-loop prediction. We omit a treatment of spectral filtering, as a full analysis of the spectral decay properties of its Hankel matrix takes us too far away from the particulars of USP.

Lemma 4.1. *Running online gradient descent on an open-loop predictor* $\hat{y}_t = \sum_{j=0}^{n} Q_j u_{t-j}$*, in an attempt to learn a linear dynamical system 1, results in regret of* $O\left(T^{\frac{3}{2}}\right)$ *if the LDS is marginally stable.*

Proof. Observe that open-loop predictors are a degenerate case of preconditioning in which our polynomial is $p_n(x) = x^n$, i.e., $c_0 = 1$ due to the monic constraint and $c_1 = ... = c_n = 0$. Accordingly, we can apply Theorem 3.3, which we previously used to show that Algorithm 3 with Chebyshev coefficients results in sublinear regret. When the system is marginally stable, since $||c||_1 = 1$ and $\max_{\lambda \in \lambda(A)} |p_n(\lambda)| = \rho(A)^n = 1$, we obtain

$$\sum_{t=1}^{T} \ell_t(Q^t) \leq C_{domain} \sqrt{d_{out}} \left(3n\sqrt{T} + (T-n)^2\right)$$

Some calculus tells us that the minimal value of n here is $n^* = T - \frac{3\sqrt{T}}{2}$, at which point the bound becomes $O\left(T^{\frac{3}{2}}\right)$, which is superlinear. □

Although this is not a lower bound, and thus does not prove that it is impossible to learn in the marginally stable case by regressing on the open-loop predictor, it demonstrates the power of USP when applying a judiciously chosen polynomial. The utility of Chebyshev lies in its ability to suppress the distant past via its minimax property while simultaneously not having the L^1 norm of its coefficients blow up fast enough so as to destroy the OGD bound. The degenerate polynomial is not able to balance this dual mandate as well and hence the analogous argument fails to prove sublinear regret in the regime of marginal stability.

Lemma 4.2. *Running online gradient descent on a closed-loop predictor* $\hat{y}_t = \sum_{j=0}^{n} Q_j u_{t-j} + \sum_{i=1}^{m} R_i y_{t-i}$*, in an attempt to learn a linear dynamical system 1, results in regret of* $O(\sqrt{T})$ *if* $n = m = d_{hidden}$*.*

Proof. For ease of notation, let $d_h = d_{hidden}$. Applying the Cayley-Hamilton Theorem, we know that A is a zero of its own characteristic polynomial, which is of degree d_h. It follows that, for some set of coefficients $\{a_i\}_{i=1}^{d_h}$,

$$A^{d_h} = \sum_{i=1}^{d_h} a_i A^{d_h - i}$$

We want to use this fact to derive a form for y_t that depends only on $\{u_t, ..., u_{t-d_h}\}$ and

$\{y_{t-1}, ..., y_{t-d_h}\}$. Unrolling the LDS yields

$$y_t = Cx_t = CA^{d_h} x_{t-d_h} + \sum_{j=1}^{d_h} CA^{j-1} Bu_{t-j}$$

$$= \sum_{i=1}^{d_h} a_i CA^{d_h - i} x_{t-d_h} + \sum_{j=1}^{d_h} CA^{j-1} Bu_{t-j}$$

where in the second line we applied the Cayley-Hamilton result. Similarly, for $1 \leq i \leq d_h - 1$, we have that

$$y_{t-i} = Cx_{t-i} = CA^{d_h - i} x_{t-d_h} + \sum_{j=1}^{d_h - i} CA^{j-1} Bu_{t-j}$$

which implies that

$$y_{t-i} - \sum_{j=1}^{d_h - i} CA^{j-1} Bu_{t-j} = CA^{d_h - i} x_{t-d_h}$$

Substituting this into the first equation, we can get rid of any explicit dependence on the hidden states:

$$y_t = \sum_{i=1}^{d_h} a_i \left(y_{t-i} - \sum_{j=1}^{d_h - i} CA^{j-1} Bu_{t-j} \right) + \sum_{j=1}^{d_h} CA^{j-1} Bu_{t-j}$$

From this, it is clear that $y_t = \sum_{j=0}^{n} N_j u_{t-j} + \sum_{i=1}^{m} M_i y_{t-i}$, where the N_j's and M_i's can be extracted from the above equation. Consequently, the best fixed closed-loop predictor is able to predict the observations with zero error, and so running OGD achieves $O(\sqrt{T})$ regret against ground truth. □

Because we are able to capture the entire past with the last d_{hidden} steps, closed-loop predictors which look d_{hidden} steps back successfully achieve sublinear regret. Of course, this still assumes that $\rho(A) \leq 1$, $||u_t||_2 \leq 1$, etc., which ensures that the bounds we get on D and G in OGD (which are omitted from the proof) are not vacuous. The problem with this technique, however, is that the hidden dimension can be arbitrarily large, especially in modern use cases like language modeling. Moreover, it is known that any nonlinear dynamical system with appropriate constraints can be approximated to arbitrary accuracy by a linear dynamical system, but with the latter having a hidden dimension size that is exponential in the hidden dimension size of the former [4]. Therefore, utilizing a closed-loop predictor in these scenarios can be impractical.

5 On the Limits of Hidden-Dimension-Free Learning

One thing to note is that Marsden and Hazan [7] claim that USP is the first technique that results in "dimension-independent sublinear regret bounds for asymmetric linear dynamical systems [whose eigenvalues have imaginary component bound above by $O(\frac{1}{\log T})$]

that are marginally stable". However, this is only true up to diagonalizability and well-conditionedness of the eigenbasis. The paper implicitly assumes that A is nice enough such that bounding the regret by condition number κ, which in principle can be arbitrarily large, is not vacuous. One can easily construct a pathological example—for instance, with a perturbed Jordan block—in which κ depends on the hidden dimension. The problem is exacerbated in the nondiagonalizable case, even for real eigenvalues.

Example 5.1. Suppose that

$$A = \begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad C = \begin{pmatrix} 0 & 0 & \cdots & 0 & 1 \end{pmatrix}$$

As A is the transpose of a $d_{hidden} \times d_{hidden}$ Jordan block, it is not diagonalizable. However, it has $\rho(A) = 0$, and thus trivially satisfies any bound on the imaginary components of its eigenvalues. Once at least d_{hidden} controls have been input into the system, we have that $x_t = [u_{t-1}, u_{t-2}, ..., u_{t-d_{hidden}}]^T$, and so $y_t = u_{t-d_{hidden}}$. In Algorithm 3, OGD works against the best fixed predictor

$$\hat{y}_t(Q^*) = -\sum_{i=1}^{n} c_i y_{t-i} + \sum_{j=0}^{n} Q_j^* u_{t-j}$$

which can only make use of the information $\mathcal{F} = \{u_t, ..., u_{t-n}\}$. Hence, if $d_{hidden} > n$—and, in USP, n is chosen to be independent of the hidden dimension—then $y_t = u_{t-d_{hidden}} \notin \mathcal{F}$. It follows that if $u_{t-d_{hidden}}$ is chosen independently of the other inputs, then predicting y_t is a futile exercise. As a result, it is not clear that running OGD provides anything meaningful, as the best fixed predictor is no longer close to the truth.

On the other hand, the paper does acknowledge the inability of hidden-dimension-free predictors to work for every matrix that is diagonalizable over $\mathbb{C}$. We include a similar counterexample as well for completeness.

Example 5.2. Consider

$$A = \begin{pmatrix} 0 & 0 & 0 & \cdots & 1 \\ 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad C = \begin{pmatrix} 0 & 0 & \cdots & 0 & 1 \end{pmatrix}$$

which is almost identical to the previous construction, except that A picks up a 1 in the top right corner, turning it into a permutation matrix. If x_0 gets initialized to 0, then after enough time, we have that $x_t = [u_{t-1} + u_{t-1-d_{hidden}} + u_{t-1-2d_{hidden}} + ..., ..., u_{t-d_{hidden}} + u_{t-2d_{hidden}} + u_{t-3d_{hidden}} + ...]^T$, and so $y_t = u_{t-d_{hidden}} + u_{t-2d_{hidden}} + u_{t-3d_{hidden}}....$ Consequently, the best fixed predictor in Algorithm 3 again stands no chance if n is chosen to be independent of the hidden dimension, as USP insists. This time, though, the failure mode of

USP is not that A is nondiagonalizable, but rather that no preconditioning polynomial can adequately squash eigenvalues which are situated too far away from the real line. The eigenvalues of the permutation matrix are the d_{hidden}^{th} roots of unity, which break the $O\left(\frac{1}{\log T}\right)$ bound on imaginary components necessary for hidden-dimension-free learnability suggested by Marsden and Hazan.

These two examples give us a concrete idea of what we can hope to accomplish with a hidden-dimension-free construction: if we lose well-conditionedness of the eigenbasis or a bound keeping the spectrum of A close to the real line, then USP becomes unusable.

6 Why USP Can Nevertheless Fail in Practice

So far, we have demonstrated why USP works (and how it fails) when trying to predict the next observation in a sequence. We will now analyze the task of multi-step prediction, describing a failure mode of USP in this scenario before detailing a solution. Since we will be shifting to training a time series model offline with a premade dataset, let us return to the version of preconditioning as defined in Algorithm 1.

Note that in order to unprecondition our prediction at time t, we need to have access to ground truth up until $t-1$, which is fine if we want to improve our next-step-ahead accuracy. However, in practice, we rarely want to predict only the next token; in this case, the standard tool becomes autoregression, in which we iteratively make our next predictions by feeding our previous ones back into the model as if they were ground truth. Applying this principle here, if, for horizon H, we want to predict $y_t, y_{t+1}, ..., y_{t+H}$ using just $\left\{u_{1:t}, y_{1:(t-1)}\right\}$, the natural idea is to slightly modify Algorithm 1 to autoregressively unprecondition. In particular, for $h \in \{0, 1, ..., H\}$,

$$\hat{y}_{t+h} = \mathcal{A}\left(u_{1:t}, y_{1:(t-1)}\right) - \sum_{i=1}^{n} c_i \hat{y}_{t+h-i}$$

where $\hat{y}_{t+h-i}$ reverts to ground truth y_{t+h-i} if $h < i$.

When testing this technique with the time series model Moirai 2.0 (described in Section 7), we found that there was severe error compounding, which made predictions significantly worse than baseline. Rather than provide a detailed catalog of experiments, we will briefly explain why this failure is reasonable, before moving onto a more robust way of applying USP to time series prediction.

For this autoregressive version of USP, assuming that the polynomial is monic, the predictions and ground truth satisfy

$$\hat{y}_{t+h} = -\sum_{i=1}^{n} c_i \hat{y}_{t+h-i} + \hat{y}_{t+h}^{prec}, \quad y_{t+h} = -\sum_{i=1}^{n} c_i y_{t+h-i} + y_{t+h}^{prec}$$

As a result, for $e_h = \hat{y}_{t+h} - y_{t+h}$ and $\mathcal{E}_h = \hat{y}_{t+h}^{prec} - y_{t+h}^{prec}$, the error evolution is given by

$$e_h = \hat{y}_{t+h} - y_{t+h} = -\sum_{i=1}^{n} c_i(\hat{y}_{t+h-i} - y_{t+h-i}) + (\hat{y}_{t+h}^{prec} - y_{t+h}^{prec}) = -\sum_{i=1}^{n} c_i e_{h-i} + \mathcal{E}_h$$

again, where $e_h = 0$ if $h < 0$, because then ground truth is available to us. We can turn this error propagation recursion into an LDS (completely separate from the underlying dynamics of the system). In a slight abuse of notation, we will for the moment redefine A and B to be specific matrices. Consider the companion matrix of the preconditioning polynomial A, in addition to the error buffer x_h:

$$A = \begin{bmatrix} -c_1 & -c_2 & \dots & -c_n \\ 1 & 0 & \dots & 0 \\ 0 & \ddots & 0 & \vdots \\ 0 & \dots & 1 & 0 \end{bmatrix}, \quad x_h = \begin{bmatrix} e_{h-1} \\ e_{h-2} \\ \vdots \\ e_{h-n} \end{bmatrix}$$

It can be easily verified that $x_{h+1} = Ax_h + B\mathcal{E}_h$, where $B = [1, 0, ..., 0]^T$. Since $e_h = 0$ for $h < 0$, we have that $x_0 = 0$. It follows that if we unroll the LDS, we obtain

$$e_h = B^T x_{h+1} = \sum_{i=0}^{h} \beta_{h-i} \mathcal{E}_i$$

where $\beta_{h-i} = B^T A^{h-i} B$. While it is difficult to make a general statement about how this affects e_h because the joint distribution of the $\mathcal{E}_i$'s is highly model-dependent, the toy example of i.i.d. $\mathcal{E}_i$'s with mean 0 and variance σ^2_{prec} is illustrative of the potential catastrophe. In this case, observe that

$$\mathbb{E}(e_h^2) = \mathrm{Var}(e_h) = \mathrm{Var}\left(\sum_{i=0}^{h} \beta_{h-i}\mathcal{E}_i\right) = \sigma^2_{prec} \sum_{i=0}^{h} \beta^2_{h-i}$$

Defining $v(h) = \sum_{i=0}^{h} \beta^2_{h-i}$, then for the degree 5 monic Chebyshev polynomial, we can calculate numerically that $v(5) \approx 4.13$, $v(20) \approx 10.34$, $v(50) \approx 11.86$, and $v(\infty) \approx 11.92$. On the other hand, for the degenerate polynomial λ^n, which corresponds to no preconditioning at all, we know that $v(h) = 1\ \forall h$. If σ^2_{base} is the per-time-step error variance for the baseline model, in order for preconditioning to decrease MSE on a prediction h steps in the future, we would need $\frac{\sigma^2_{prec}}{\sigma^2_{base}} < \frac{1}{v(h)}$, which is a tall task.

One way to understand this is that, by the definition of β_{h-i}, the linear combination that produces e_h has coefficients whose asymptotic decay is exponential in $\rho(A)$. The problem, however, is that the characteristic polynomial of A is precisely our preconditioning polynomial $p_n(\lambda) = \lambda^n + c_1\lambda^{n-1} + ... + c_n$, which has roots

$$\lambda_k = \cos\left(\frac{(2k-1)\pi}{2n}\right), \quad k = 1, 2, ..., n$$

when specialized to Chebyshev. This implies that $\rho(A) = \cos\left(\frac{\pi}{2n}\right)$, which is always less than 1—and so we do get decay—but not by much. For $n = 5$, which Marsden and Hazan claim is empirically best, we have that $\rho(A) = \cos\left(\frac{\pi}{10}\right) \approx 0.95$. Because the coefficients stay relatively large, we would need destructive interference between the preconditioned model's errors at different time steps to ensure that MSE does not blow up, but the model is not explicitly trained to achieve this goal.

On the other hand, the degenerate preconditioning polynomial $p_n(\lambda) = \lambda^n$ has $c_1 = ... = c_n = 0$, and so its companion matrix has spectral radius $\rho(A) = 0$. In fact, it must be that $\beta_h = 0 \ \forall h \geq 1$ because if we do not precondition, the model's error at a particular time should not explicitly depend on the values of other errors, beyond the inherent correlation of outputs from the same model.

The large spectral radius of A for the Chebyshev polynomial is not incidental. From the proof of Lemma 3.1, the form of the nth degree monic Chebyshev polynomial $M_n(x)$ that allows us to prove the minimax property is

$$M_n(x) = \frac{\cos(n \arccos(x))}{2^{n-1}}$$

for $x \in [-1, 1]$. The cosine enables the squashing of all values on $[-1, 1]$, but it is also exactly what causes the roots to cluster near 1. Therein lies the irony of USP—the same Chebyshev property that suppresses the distant past in the proof of Theorem 3.3 hurts our ability to autoregressively unprecondition when attempting to predict multiple steps ahead.

7 An Empirical Solution[1]

In view of these issues with naively applying preconditioning, we will propose an adaptation, called a "preconditioning hint", aimed at bridging the gap between theory and practice. As we have seen, performing unpreconditioning during inference by plugging back in our previous predictions as placeholders for ground truth likely destroys any gains that we may obtain by predicting in the preconditioned space. The motivation behind the preconditioning hint is accordingly to find a way for the model to reap the benefits of USP without needing this extra autoregressive step.

At a high level, we accomplish this by concatenating an input sequence with the difference between its preconditioned counterpart and itself (the hint) and feeding this into the model instead, with the goal of predicting the nonpreconditioned output. During training, we also perform dropout regularization on the hint, meaning that we randomly zero out the hint—but not the original input—with a fixed probability. This ensures that the model does not overrely on it. Otherwise, we keep the architecture exactly the same, making the hint quite universal and in the spirit of Marsden and Hazan.

As a baseline, we use a recreated version of Moirai 2.0 [6], a state-of-the-art time series foundation model created by Salesforce based on the transformer architecture. We train on the LOTSA dataset [8] and test on the GIFT-EVAL benchmark [1], both of which include a variety of data across different domains, frequencies, and time horizons, and are intended for zero-shot time series models such as Moirai 2.0.

While we leave a detailed discussion of the intricacies of the model architecture to the Moirai 2.0 paper, one important note is that before the data is fed into the transformer, it is split into patches of constant size (we use a patch size of 16), from which point on the model operates

[1] Code available at `https://github.com/jerryhan60/spectral-time-series`.

entirely on a patch level. In other words, for a sequence $y_0, y_1, ..., y_t, ...$, the transformer sees $[y_0, ..., y_{15}], [y_{16}, ..., y_{31}], ...$ instead of $y_0, y_1, ...$ individually, and outputs its predictions in patch form. Therefore, we perform preconditioning at a patch level as well. Observe that this is equivalent to preconditioning at a time-step level with a stride equal to patch size, i.e., for each time step t in a patch, we append the hint

$$h_t = y_t^{prec} - y_t = \sum_{k=0}^{n} c_k y_{t-ks} - y_t$$

to the patch, where $s = 16$ is the stride (and patch) size and $\{c_k\}$ is the set of coefficients of the preconditioning polynomial of our choice. Concretely, then, each patch becomes

$$[y_t, ..., y_{t+15}, h_t, ..., h_{t+15}]$$

disregarding the per-time-step binary mask marking if any data is missing, which is also appended to the patch in the actual model.

Below are our main results. All runs are trained with the same hyperparameters—most notably, all for 10K steps. At times, we refer to our primary model as HD10 (for **H**int **D**ropout **10**%), while referring to the baseline Moirai 2.0 model as BL. HD10 uses the degree 4 Chebyshev polynomial, 10% dropout regularization, and stride = patch size = 16. As the names suggest, the zero hint model sets the hint equal to zero and the duplicate channel passes in the sequence a second time as the hint. The metric that we use is geometric mean MASE (Mean Absolute Scaled Error) over the 97 GIFT-Eval configurations, where we scale by the seasonal naive predictor for each configuration. The seasonal naive predictor simply predicts any future value to be the most recently observed past value at the same point in a previous season; for instance, for hourly weather data, it would predict the temperature tomorrow at 8am to be the same as the temperature today at 8am.

For ablations on our design choices for HD10, see Appendix A. In short, they all appear to be optimal when holding everything else fixed.

Model	Norm. MASE	Δ vs baseline	HD10 wins / 97
Baseline	0.862	—	69/97
Zero hint	0.858	-0.5%	73/97
Duplicate channel	0.865	+0.4%	79/97
HD10 (Chebyshev $d = 4$)	**0.838**	**-2.8%**	**—**

Figure 1: Main result (across 5 seeds).

In Figure 1, our Chebyshev preconditioning hint realizes a nontrivial gain of 2.8% relative to the original Moirai 2.0. Moreover, the zero hint and duplicate channel both perform around baseline, indicating that elongating the patch on its own does not achieve anything. That HD10 wins against baseline on average 69/97 times over 5 seeds ($p < 2 \times 10^{-5}$ under the one-sided binomial test) suggests that the gain is not noise. Rather, the hint provides a useful inductive bias to the transformer, which learns to incorporate the Chebyshev structure into its predictions. Figure 2a goes further, telling us that Chebyshev is not an arbitrary

nontrivial hint choice—it outperforms the Legendre polynomial, in addition to high-pass and low-pass filters. Note that our method increases the parameter count of the model by a negligible 0.11%, significantly less than our 2.8% gain, implying that our improvement is not simply from boosting model size.

With that being said, our approach does contain limitations. Due to compute constraints, some of our design choices were only verified on one seed, and it is possible that they are suboptimal when averaged over many seeds. For the same reason, we were only able to test the preconditioning hint on the small version of Moirai 2.0. In the Moirai 2.0 paper, the small model actually outperformed the larger ones, but it is nevertheless possible that the hint fails to work on larger versions. Additionally, Moirai 2.0 is not entirely open source, and large chunks of the data Salesforce used to train it are proprietary. As a result, our version of Moirai 2.0 performs worse than the original one, and it is possible that the hint loses its edge when the model is fed more and better training data. Finally, it is possible that it fails on different time series foundation models or on different datasets for training and testing.

Looking forward, another potential approach to incorporate preconditioning into time series prediction is to work more directly with the framework introduced in Section 6. While the preconditioning hint sidesteps the problems with naive autoregressive unpreconditioning entirely, this may have been an unnecessarily drastic step. In particular, perhaps it is possible to engineer a universal polynomial with a Chebyshev-like minimax property that does a better job of minimizing the error compounding in an autoregressive rollout. Combining this with a mechanism to make the preconditioned model aware that autoregressive unpreconditioning will occur during inference, such as a manipulation of the loss function, may work. This was a direction that we originally pursued, with somewhat promising results, but we opted to dive more deeply into the hint instead.

Applying the hint to other related domains, such as language modeling, is another potentially fruitful area of research, which we leave to future work as well.

8 Conclusion

We began this paper by stepping through the theoretical contributions of Marsden and Hazan with USP. It succeeds (read: achieves sublinear regret) for diagonalizable linear dynamical systems whose eigenvalues are in the slice of the complex plane with imaginary components bounded by $O\left(\frac{1}{\log T}\right)$, though our proofs worked with diagonalizability over $\mathbb{R}$ for simplicity. On the contrary, spectral filtering, as originally presented, requires diagonalizability over $\mathbb{R}$, closed-loop predictors require a parameterization dependent on the hidden dimension, and open-loop predictors fail in the case of marginal stability, the latter two of which we looked at in detail. We then brought two examples to illustrate the failure modes of USP when our diagonalizability assumptions break.

Next, we shifted from theory to practice, noting that predicting only the next element in a sequence is usually not the goal in time series prediction. We provided some intuition as to why naive autoregressive unpreconditioning fails with the Chebyshev polynomial, before

moving to an alternative solution called the preconditioning hint.

The hint demonstrated that preconditioning can be applied to time series prediction, notwithstanding that we usually do not have ground truth to unprecondition the model output. While the gains were a modest 2.8%, they were statistically significant and not merely a result of increased model capacity. Furthermore, the Chebyshev polynomial resulted in better accuracy than other hint candidates, suggesting that it does have a structure that helps with learning. Even though our tests were not comprehensive, they should be viewed as indicative that the line of research introduced by USP is promising.

A Ablation Studies

Below are our ablations. All runs are trained for 10K steps, and use the degree 4 Chebyshev polynomial, 10% dropout regularization, and stride = patch size = 16, unless the experiment is explicitly ablating on one of these design choices. Note that some experiments have 5 seeds of data, while others only have information from seed 0. This is a result of compute constraints, and future runs are needed to check other seeds.

Filter basis	Norm. MASE	Δ vs baseline
Baseline	0.867	—
Chebyshev ($d = 4$)	**0.823**	**-5.0%**
Legendre ($d = 4$)	0.840	-3.1%
Legendre ($d = 6$)	0.840	-3.1%
EMA	0.833	-3.9%
Differencing ($d = 4$)	0.857	-1.1%

(a) Filter basis (seed 0).

Degree	Norm. MASE	Δ vs baseline
2	0.850	-1.3%
3	0.842	-2.3%
4	**0.838**	**-2.8%**
5	0.842	-2.3%
6	0.840	-2.5%
7	0.847	-1.7%
8	0.841	-2.4%

(b) Chebyshev degree (5 seeds).

Stride	Norm. MASE	Δ vs baseline
4	0.841	-3.0%
8	0.852	-1.7%
16	**0.823**	**-5.0%**
32	0.848	-2.1%

(c) Hint stride (seed 0, still patch size = 16).

Dropout	Norm. MASE	Δ vs baseline
0%	0.833	-3.9%
10%	**0.823**	**-5.0%**
20%	0.842	-2.8%
30%	0.856	-1.2%

(d) Hint dropout (seed 0).

Seed	BL 10K	HD10 10K	Δ 10K	BL 100K	HD10 100K	Δ 100K
0	0.867	0.823	-5.0%	0.909	0.835	-8.2%
1	0.850	0.847	-0.4%	0.892	0.840	-5.8%
2	0.863	0.845	-2.1%	0.863	0.859	-0.5%
7	0.868	0.832	-4.1%	0.864	0.845	-2.2%
42	0.861	0.843	-2.1%	0.864	0.843	-2.4%
mean	**0.862**	**0.838**	**-2.8%**	**0.878**	**0.844**	**-3.9%**

(e) Training scale (5 seeds).

Figure 2: Ablation studies. Top row: filter basis and Chebyshev degree. Middle row: hint stride and hint dropout. Bottom: training scale.

In Figure 2a, the Legendre polynomial is presented in Marsden and Hazan as a potential alternative to Chebyshev. EMA is an exponential moving average of the previous 4 patches, and differencing is a fourth-order finite difference with binomial coefficients. The former is a low-pass filter, while the latter is a high-pass filter.

In Figure 2c, we are testing whether preconditioning with a stride that does not align with the patch size of 16 can potentially capture cross-patch interactions better than the natural stride choice equal to patch size.

Figure 2e suggests that our gains are not a matter of undertraining the models. Moreover, the performance of BL degrades faster than HD10 at 100K steps, indicating that it begins overfitting the training data earlier on.

References

[1] Taha Aksu, Gerald Woo, Juncheng Liu, Xu Liu, Chenghao Liu, Silvio Savarese, Caiming Xiong, and Doyen Sahoo. Gift-eval: A benchmark for general time series forecasting model evaluation, 2024.

[2] George E. P. Box and Gwilym M. Jenkins. *Time Series Analysis: Forecasting and Control*. Holden-Day Series in Time Series Analysis. Holden-Day, 1976.

[3] Elad Hazan. Introduction to online convex optimization, 2023. arXiv:1909.05207v3.

[4] Elad Hazan and Karan Singh. Introduction to online control, 2026. arXiv:2211.09619v7.

[5] Elad Hazan, Karan Singh, and Cyril Zhang. Learning linear dynamical systems via spectral filtering, 2017.

[6] Chenghao Liu, Taha Aksu, Juncheng Liu, Xu Liu, Hanshu Yan, Quang Pham, Silvio Savarese, Doyen Sahoo, Caiming Xiong, and Junnan Li. Moirai 2.0: When less is more for time series forecasting, 2026. arXiv:2511.11698v3.

[7] Annie Marsden and Elad Hazan. Universal sequence preconditioning, 2025. arXiv:2502.06545v2.

[8] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers, 2024.